

Claro. Abaixo está um **checklist profissional completo para desenvolver um aplicativo utilizando uma IA como agente de desenvolvimento**. Ele foi pensado para evitar os principais problemas de projetos em que a IA gera código rapidamente, mas deixa inconsistências de arquitetura, segurança, banco de dados ou regras de negócio.

## **CHECKLIST PROFISSIONAL — DESENVOLVIMENTO DE APP COM IA**

**Regra de ouro:** a IA deve ser tratada como **desenvolvedora assistente**, não como autoridade sobre as regras do sistema. As regras, arquitetura e critérios de aceitação devem ser definidos e validados por você.

---

### **FASE 0 — DEFINIÇÃO DO PROJETO**

#### **Identidade do projeto**

- Nome do aplicativo definido
- Objetivo principal definido
- Problema que o aplicativo resolve documentado
- Público-alvo definido
- Plataforma definida:
  - Web
  - Android
  - iOS
  - Desktop
  - PWA
- Necessidade de funcionamento offline definida
- Necessidade de múltiplos usuários definida
- Necessidade de sincronização definida

#### **Documento principal**

Crie um arquivo:

PROJECT\_SPEC.md

Contendo:

1. Objetivo
2. Público-alvo
3. Funcionalidades

4. Regras de negócio
  5. Usuários
  6. Permissões
  7. Dados
  8. Integrações
  9. Segurança
  10. Requisitos técnicos
  11. Critérios de aceitação
  12. Restrições
- 

## FASE 1 — REQUISITOS

Para **cada funcionalidade**, documente:

Nome:

Objetivo:

Quem pode utilizar:

Dados necessários:

Resultado esperado:

Regras:

Exceções:

Erros possíveis:

Critérios de aceitação:

### **Exemplo**

FUNCIONALIDADE: Criar compra

Entrada:

Valor

Data

Cartão

Número de parcelas

Processamento:

Registrar compra

Gerar parcelas

Associar parcelas às respectivas faturas

Resultado:

Compra cadastrada

Parcelas criadas

Fatura atualizada

Limite atualizado

### **Checklist**

- Requisitos funcionais
  - Requisitos não funcionais
  - Regras de negócio
  - Casos de erro
  - Casos extremos
  - Critérios de aceitação
  - Fluxos de usuário
- 

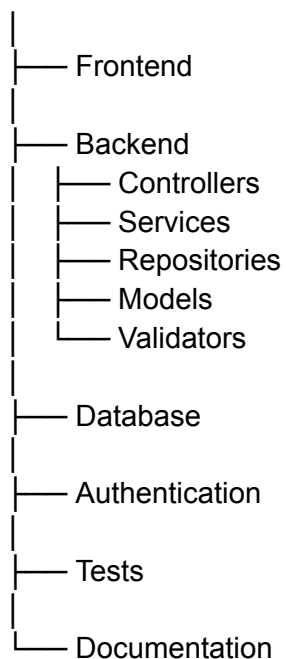
## FASE 2 — ARQUITETURA

Antes de pedir para a IA escrever código:

- Arquitetura definida
- Frontend definido
- Backend definido
- Banco definido
- API definida
- Autenticação definida
- Autorização definida
- Sistema de logs definido
- Sistema de tratamento de erros definido
- Sistema de backup definido

Uma estrutura típica:

APP



## FASE 3 — BANCO DE DADOS

Essa fase merece atenção especial.

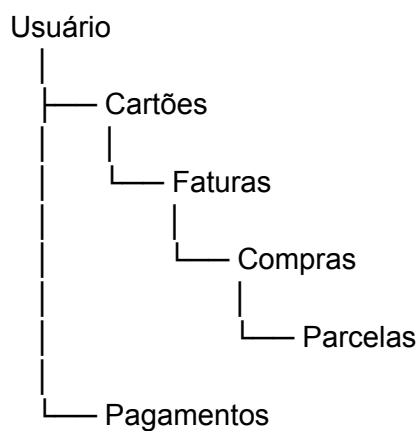
### Antes de programar

- Entidades identificadas
- Relacionamentos definidos
- Chaves primárias
- Chaves estrangeiras
- Índices
- Constraints
- Campos obrigatórios
- Campos opcionais
- Regras de exclusão
- Histórico/auditoria

### Pergunte sempre:

"Se esse registro for excluído, o que acontece com os registros relacionados?"

### Exemplo



## FASE 4 — DINHEIRO E CÁLCULOS

Se o aplicativo trabalha com dinheiro:

### NÃO confie cegamente em:

float  
double

Prefira:

Decimal

ou valores inteiros em centavos:

R\$ 10,50

→ 1050 centavos

## Checklist

- Arredondamento definido
  - Casas decimais definidas
  - Moeda definida
  - Juros definidos
  - Taxas definidas
  - Descontos definidos
  - Multas definidas
  - Parcelamento definido
  - Estorno definido
  - Cancelamento definido
- 

## FASE 5 — DATAS E HORÁRIOS

Datas são uma das maiores fontes de bugs.

Defina:

- Fuso horário
- Formato de armazenamento
- Formato de exibição
- Data de vencimento
- Dias úteis
- Finais de semana
- Feriados
- Ano bissexto
- Virada de mês
- Virada de ano

Nunca deixe a IA "assumir" uma regra importante.

---

## FASE 6 — SEGURANÇA

## Autenticação

- Login
- Logout
- Recuperação de senha
- Expiração de sessão
- MFA/2FA quando necessário
- Hash seguro de senha

## Autorização

- Roles
- Permissions
- Controle no frontend
- Controle no backend

**Importante:** esconder um botão no frontend **não é segurança**.

O backend também precisa verificar a permissão.

## Dados

- HTTPS
- Criptografia quando necessária
- Secrets fora do código
- API Keys protegidas
- Tokens protegidos
- Logs sem informações sensíveis
- Backup protegido

---

# FASE 7 — IA COMO AGENTE DE DESENVOLVIMENTO

Aqui está uma das partes mais importantes.

### Não faça:

"Construa meu aplicativo inteiro."

### Faça:

Analise o projeto.

Não altere nenhum arquivo.

Identifique:

- problemas
- dependências
- riscos
- inconsistências
- funcionalidades faltantes

Depois apresente um plano de implementação.

Depois:

Implemente somente a etapa 1.

Não altere funcionalidades existentes.

Não altere contratos de API sem autorização.

Crie os testes correspondentes.

Ao terminar:

1. liste os arquivos alterados;
  2. explique cada alteração;
  3. execute os testes;
  4. informe qualquer problema encontrado.
- 

## FASE 8 — CONTROLE DE ALTERAÇÕES

Use Git.

Fluxo recomendado:

ESTÁVEL



Branch



IA implementa



Testes



Code Review



Merge

↓  
ESTÁVEL

### Antes de uma alteração grande:

- Commit realizado
- Backup realizado
- Branch criada

### Depois:

- Testes executados
  - Diferenças revisadas
  - Arquivos alterados conferidos
  - Commit realizado
- 

## FASE 9 — TESTES

Não peça apenas:

"Teste o aplicativo."

Peça testes específicos.

### Unitários

Entrada → processamento → resultado

### Integração

Frontend

↓

API

↓

Banco

### E2E

Usuário

↓

Login

↓

Cadastro

↓

Operação

↓

Resultado

---

## FASE 10 — CASOS EXTREMOS

Para cada funcionalidade:

### **Teste normal**

R\$ 100,00

### **Teste mínimo**

R\$ 0,01

### **Teste máximo**

R\$ 999.999.999,99

### **Teste inválido**

R\$ -100

### **Teste vazio**

valor = NULL

### **Teste concorrente**

Usuário A altera

Usuário B altera simultaneamente

---

## FASE 11 — API

Para cada endpoint:

- Método HTTP
- URL
- Autenticação
- Permissão
- Entrada
- Validação
- Resposta
- Código HTTP
- Tratamento de erro
- Rate limit
- Logs

Exemplo:

POST /api/compras

Entrada:

```
{  
  "valor": 1000,  
  "parcelas": 10  
}
```

Saída esperada:

201 Created

---

## FASE 12 — TRATAMENTO DE ERROS

Nunca permita que o aplicativo simplesmente:

ERROR

Crie mensagens adequadas.

Exemplo:

Não foi possível registrar a compra.

Código: PURCHASE\_CREATE\_FAILED

E no log:

timestamp  
usuário  
endpoint  
erro  
stack trace  
request id

---

## FASE 13 — INTERFACE

**Cada tela**

- Loading
- Sucesso

- Erro
- Estado vazio
- Validação
- Responsividade
- Acessibilidade
- Navegação
- Feedback visual

Não teste somente o "caminho feliz".

---



## FASE 14 — PERFORMANCE

Verifique:

- Tempo de carregamento
- Consultas ao banco
- Índices
- Cache
- Imagens
- APIs
- Paginação
- Processamentos pesados

Pergunte à IA:

"Quais partes deste sistema podem apresentar problemas de performance quando tivermos 100 mil usuários?"

Depois:

"E com 1 milhão?"

Isso ajuda a encontrar problemas arquiteturais antecipadamente.

---



## FASE 15 — PRIVACIDADE E LGPD

Se o aplicativo tratar dados pessoais no Brasil:

- Dados coletados documentados
- Finalidade definida
- Necessidade de cada dado avaliada
- Política de privacidade

- Consentimento quando aplicável
- Exclusão de dados
- Exportação de dados quando aplicável
- Controle de acesso
- Retenção de dados
- Logs
- Incidentes de segurança

Para sistemas que tratam dados financeiros ou pessoais em escala, vale fazer uma revisão jurídica específica.

---

## FASE 16 — BACKUP

Tenha:

Backup do banco

+

Backup dos arquivos

+

Backup das configurações

E principalmente:

**Teste o restore.**

Um backup que nunca foi restaurado/testado **não deve ser considerado plenamente confiável.**

---

## FASE 17 — OBSERVABILIDADE

Depois de publicado:

- Logs
- Monitoramento
- Métricas
- Alertas
- Erros
- Performance
- Disponibilidade
- Auditoria

Você precisa conseguir responder:

"O que aconteceu às 14:32 quando esse usuário tentou realizar essa operação?"

---

## FASE 18 — AUDITORIA

Para operações importantes:

Quem?  
Quando?  
O quê?  
Antes?  
Depois?  
De onde?

Exemplo:

Usuário: 154  
Data: 25/08/2026 17:32  
Operação: ALTEROU\_FATURA  
Antes: R\$ 1.200  
Depois: R\$ 1.500

Isso é extremamente importante em sistemas financeiros, administrativos e empresariais.

---

## FASE 19 — DEPLOY

Antes de colocar em produção:

- Variáveis de ambiente
  - Secrets
  - Banco de produção
  - Backup
  - HTTPS
  - Domínio
  - CORS
  - Rate limiting
  - Logs
  - Monitoramento
  - Migrações
  - Rollback
-

# FASE 20 — CHECKLIST DE PRODUÇÃO

Antes do lançamento:

## Bloqueadores

- Nenhum erro crítico
- Nenhuma vulnerabilidade crítica conhecida
- Backup funcionando
- Restore testado
- Autenticação funcionando
- Permissões funcionando
- Banco consistente
- Cálculos validados
- Logs funcionando

## Importantes

- Performance aceitável
- Interface responsiva
- Tratamento de erros
- Documentação
- Monitoramento

## Melhorias futuras

- Novos recursos
- Otimizações
- Melhorias de UX
- Automação



## CHECKLIST ESPECÍFICO PARA A IA

Antes de aceitar código gerado pela IA, pergunte:

- ☐ Você entendeu os requisitos?
- ☐ Quais são suas suposições?
- ☐ Quais arquivos serão alterados?
- ☐ Existe risco de quebrar funcionalidades existentes?
- ☐ Existem alterações no banco?
- ☐ Existem alterações na API?

- ☐ Existem problemas de segurança?
- ☐ Existem problemas de concorrência?
- ☐ Existem problemas de performance?
- ☐ Existem casos extremos não tratados?
- ☐ Foram criados testes?
- ☐ Os testes foram executados?
- ☐ O código segue a arquitetura existente?
- ☐ Existem dependências novas?
- ☐ Alguma funcionalidade foi inventada?

**E uma regra particularmente importante:**

**Nunca permita que a IA invente uma regra de negócio silenciosamente.**

Se ela encontrar algo como:

"Não está definido o que acontece quando..."

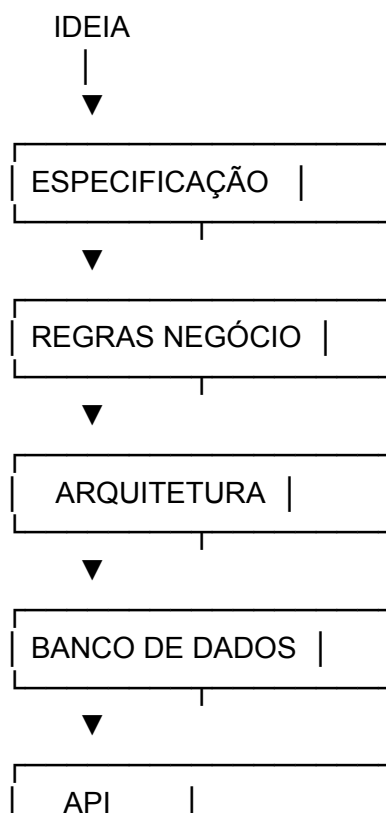
Ela deve **parar e perguntar**, ou registrar a decisão como uma pendência.

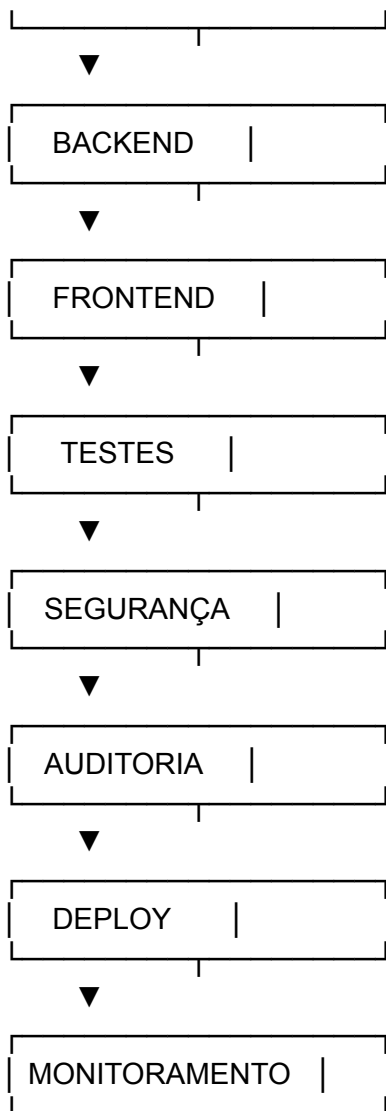
---



## FLUXO PROFISSIONAL COMPLETO

Eu recomendo usar este fluxo:





## ★ E eu acrescentaria uma regra ao seu processo

Como você está usando IA para desenvolver projetos, crie um arquivo chamado:

AI\_RULES.md

Nele coloque as regras que **toda IA que trabalhar no projeto deve obedecer**:

1. Não alterar regras de negócio sem autorização.
2. Não remover funcionalidades existentes sem autorização.
3. Não criar dados fictícios para substituir dados reais.
4. Não expor secrets, tokens ou senhas.
5. Não alterar o banco sem criar migração.

6. Toda nova funcionalidade deve possuir testes.
7. Toda correção deve possuir teste de regressão quando aplicável.
8. Não modificar APIs existentes sem documentar a alteração.
9. Não adicionar dependências sem justificar.
10. Antes de grandes alterações, criar um plano.
11. Informar todos os arquivos modificados.
12. Nunca considerar uma implementação concluída sem executar os testes.
13. Em caso de ambiguidade de regra de negócio, solicitar decisão.
14. Preservar compatibilidade com funcionalidades existentes.
15. Priorizar segurança, consistência e integridade dos dados.

**Isso transforma o projeto de "uma IA escrevendo código" em um processo de engenharia de software assistido por IA.**

Se o seu objetivo é construir um **app financeiro/cartão**, eu iria além: faria uma **checklist específica para sistemas financeiros**, cobrindo **fatura, fechamento, vencimento, compra, parcelamento, juros, multa, rotativo, pagamento parcial, estorno, limite, antecipação de parcelas e conciliação**, porque aí existem várias regras que precisam ser definidas antes de deixar a IA programar.